

SISTEMA DE GESTIÓN CON LISTA, COLA Y PILA

Caso 6: Mesa de Ayuda TI / TicketSoporte

Universidad: Universidad de Cartagena

Asignatura: Estructuras de Datos

Nombre de la actividad: Actividad II - SIMA-PESAD

Nombre del estudiante: Jesus David Campo Yunes

Código del estudiante: 7502410028

1. Logros de Aprendizaje

Estructuras de datos

Durante el desarrollo de esta actividad comprendí de manera práctica qué son las estructuras de datos y para qué sirven. Una estructura de datos es una forma organizada de almacenar y gestionar información en memoria, de modo que pueda ser accedida y modificada eficientemente. Aprendí que no existe una estructura universal: cada una tiene un propósito específico y elegir la correcta impacta directamente en la calidad y eficiencia de la solución.

Diferencias entre Lista, Cola y Pila

Aunque las tres estructuras almacenan colecciones de elementos mediante nodos enlazados, se diferencian radicalmente en el modo de acceso y manipulación:

- **Lista:** acceso libre a cualquier posición. Permite agregar, eliminar y recorrer en ambos sentidos. Ideal como registro general.
- **Cola (FIFO):** el primer elemento en entrar es el primero en salir. Modela perfectamente la atención por orden de llegada.
- **Pila (LIFO):** el último elemento en entrar es el primero en salir. Permite implementar historial y deshacer operaciones.

Nodos enlazados

Aprendí que las tres estructuras se construyen sobre la clase Nodo, que encapsula un dato genérico (Object) y referencias al nodo siguiente y anterior. Esto elimina la necesidad de arreglos de tamaño fijo: la lista crece dinámicamente en memoria. Comprendí cómo manipular referencias para insertar y eliminar nodos sin mover datos físicamente.

Dificultades encontradas

La principal dificultad fue implementar la cancelación de un ticket pendiente, ya que la cola no

permite acceso directo al centro. Resolví el problema usando una cola auxiliar: se vacía la cola original omitiendo el elemento a cancelar, y luego se restaura. También fue desafiante mantener la consistencia del estado del ticket a través de las tres estructuras simultáneamente.

Programación Orientada a Objetos (POO)

Aplicué encapsulamiento (atributos privados con getters/setters), abstracción (cada clase tiene una responsabilidad clara), composición (Lista, Cola y Pila contienen Nodos) y polimorfismo implícito mediante el tipo genérico Object. Implementé los métodos equals() y hashCode() para garantizar una búsqueda correcta por código de ticket.

Validaciones y manejo de errores

Aprendí a validar entradas en el constructor de TicketSoporte lanzando IllegalArgumentException con mensajes descriptivos. Las operaciones críticas (desencolar, desapilar) lanzan RuntimeException cuando la estructura está vacía. En el menú principal usé bloques try-catch para capturar errores de formato numérico al leer opciones del usuario.

Flujo de datos y Git

Comprendí cómo el mismo objeto TicketSoporte fluye por las tres estructuras: nace en la Lista, espera en la Cola, y descansa en la Pila. Git me permitió versionar el proyecto con commits progresivos, documentando cada etapa del desarrollo: estructura base, lógica de menú, validaciones y pruebas finales.

2. Informe de Desarrollo

2.1 Descripción funcional del sistema

La aplicación simula la operación de una Mesa de Ayuda TI: un departamento de soporte técnico que recibe reportes de fallas de los usuarios, los atiende en orden de llegada y mantiene un historial de lo procesado.

El sistema permite: registrar tickets de soporte con código, usuario, descripción de falla y prioridad; visualizar todos los tickets, los pendientes y el historial; procesar el siguiente ticket de la cola; buscar por código; cancelar un ticket pendiente; y deshacer el último procesamiento devolviendo el ticket a la cola.

2.2 Componentes desarrollados

- **Nodo.java:** Nodo doblemente enlazado genérico. Almacena un Object y mantiene referencias al nodo siguiente y anterior. Es el bloque fundamental de las tres estructuras.
- **Lista.java:** Lista doblemente enlazada. Actúa como registro general de todos los tickets. Permite agregar al inicio/fin/posición, eliminar, buscar y recorrer en ambas direcciones.
- **Cola.java:** Estructura FIFO. Gestiona los tickets pendientes de atención. El primer ticket registrado es el primero en procesarse. Implementa encolar(), desencolar() y peek().

- **Pila.java:** Estructura LIFO. Almacena el historial de tickets procesados. Permite deshacer el último procesamiento desapilando y devolviendo el ticket a la cola.
- **TicketSoporte.java:** Entidad principal del caso. Encapsula: codigoTicket, nombreUsuario, descripcionFalla, prioridad y estado. Implementa equals() comparando por codigoTicket.
- **SistemaMesaAyudaTI.java:** Clase principal con el método main(). Contiene las tres estructuras como atributos estáticos, el menú interactivo y todos los casos de uso.

Relación entre clases

SistemaMesaAyudaTI contiene instancias de Lista, Cola y Pila. Las tres estructuras contienen instancias de Nodo. Cada Nodo almacena un Object que en la práctica es siempre un TicketSoporte. El flujo es unidireccional: el ticket nace en la Lista, viaja a la Cola, y termina en la Pila (o regresa a la Cola si se deshace la operación).

2.3 Estructuras de datos implementadas

Lista doblemente enlazada

La Lista mantiene dos punteros: cabeza (primer nodo) y cola (último nodo). Al agregar al final, el nodo nuevo se enlaza al nodo cola actual y se actualiza el puntero. Para recorrer adelante se sigue la referencia siguiente; para recorrer atrás se sigue anterior. En este sistema la Lista actúa como registro inmutable de todos los tickets: un ticket registrado nunca se elimina de la lista, solo cambia su estado.

Métodos implementados en Lista: agregar(), agregarAlInicio(), agregarEnPosicion(), eliminarPrimero(), eliminarUltimo(), eliminarEnPosicion(), buscarDato(int), buscarDato(Object), contiene(), cuentaElementos(), limpiar(), mostrarAdelante(), mostrarAtras().

Fragmento clave (agregar al final)

```
Nodo nuevo = new Nodo(dato);
if (cabeza == null) {
    cabeza = nuevo;
    cola = nuevo;
} else {
    nuevo.setAnterior(cola);
    cola.setSiguiente(nuevo);
    cola = nuevo;
}
tamano++;
```

Cola FIFO

La Cola mantiene punteros frente (primer elemento a despachar) y fin (último encolado). Al encolar se enlaza al fin; al desencolar se avanza el frente. Esto garantiza que el primer ticket reportado sea siempre el primero atendido, modelando fielmente una fila de espera real en el

departamento de soporte.

Fragmento clave (cancelar con cola auxiliar)

```
Cola colaAuxiliar = new Cola();
while (!colaPendientes.esVacia()) {
    TicketSoporte actual = (TicketSoporte)
colaPendientes.desencolar();
    if (!actual.getCodigoTicket().equalsIgnoreCase(codigo)) {
        colaAuxiliar.encolar(actual);
    }
}
while (!colaAuxiliar.esVacia()) {
    colaPendientes.encolar(colaAuxiliar.desencolar());
}
```

Pila LIFO

La Pila usa un único puntero tope. Al apilar, el nuevo nodo apunta al tope actual y se convierte en el nuevo tope. Al desapilar se retorna el dato del tope y se avanza al siguiente. Esto implementa el historial de procesados y el mecanismo de deshacer: la última operación realizada siempre es la primera disponible para revertirse.

Fragmento clave (deshacer procesamiento)

```
TicketSoporte ultimo = (TicketSoporte)
historialProcesados.desapilar();
ultimo.setEstado("PENDIENTE");
colaPendientes.encolar(ultimo);
```

2.4 Flujo del sistema

1. **Registrar un ticket:** El usuario ingresa nombre, descripción y prioridad. Se genera un código automático (TKT-0001, TKT-0002...). Se crea el objeto TicketSoporte con estado PENDIENTE. Se llama listaGeneral.agregar(ticket) y colaPendientes.encolar(ticket).
2. **Ticket en la cola:** El ticket espera en la cola FIFO. Si llegan más tickets, se acumulan detrás. La operación peek() permite ver el próximo a procesar sin eliminarlo.
3. **Procesar siguiente:** Se llama colaPendientes.desencolar(). El ticket sale de la cola, su estado cambia a PROCESADO y se llama historialProcesados.apilar(ticket).
4. **Historial:** La pila mantiene los tickets procesados. El más reciente está en el tope (visible con peek()).
5. **Búsqueda:** Se crea un TicketSoporte auxiliar con el código buscado y se llama listaGeneral.buscarDato(auxiliar). El método equals() compara por codigoTicket.
6. **Cancelar:** Se usa una cola auxiliar para reconstruir la cola omitiendo el ticket cancelado. El estado del ticket se actualiza a CANCELADO en el objeto.
7. **Deshacer:** Se desapila el último procesado, se cambia su estado a PENDIENTE y se

encola nuevamente al final de la cola de pendientes.

3. Enlace del Repositorio GitHub

El repositorio público del proyecto está disponible en:

<https://github.com/tab-Jesus/ACt-2-estructura-de-datos>

Requisito	Cumplimiento
Commits progresivos durante el desarrollo	✓
Mensajes de commit claros y descriptivos	✓
No se hizo un único commit masivo	✓
Repositorio público y completo	✓
Historial evidencia el proceso de construcción	✓

4. Enlace del Video de Sustentación

El video de sustentación está disponible en:

URL: <https://youtu.be/CZrTwk6D1nk>

Contenido demostrado en el video:

- Registro de tickets con validación de datos de entrada.
- Visualización de la lista general (todos los tickets).
- Demostración del orden FIFO en la cola de pendientes.
- Procesamiento del siguiente ticket y actualización del historial (Pila).
- Búsqueda de ticket por código usando equals().
- Cancelación de un ticket pendiente mediante cola auxiliar.
- Deshacimiento del último procesamiento (LIFO).
- Manejo de errores: cola vacía, pila vacía, código no encontrado.
- Explicación técnica del código: nodos, referencias, FIFO y LIFO.